

PriorityKV: A Measurement Study of Structure-Aware KV Retention and Mixed-Precision Storage for Long Agent Traces

Arush Sharma¹ and Anupam Rawat²

¹Indian Institute of Technology (Indian School of Mines) Dhanbad

²Indian Institute of Technology Bombay

Abstract

Long language-model agent traces contain heterogeneous state—tool schemas, system instructions, superseding constraints, identifiers, dialogue, and filler—whose tokens occupy the key-value (KV) cache with very different behavioral consequences when lost. We study whether cache allocation should use this application-visible trace structure, and we report the results as a scoped measurement study on Qwen3-8B (H200). We introduce PriorityBench-A, a locked 240-example benchmark with deterministic scorers covering tool-schema conformance, instruction supersession, and multi-turn state at 8k, 16k, and 32k context strata. Under matched-budget eviction stress at a 25% keep budget, a role-blind sink-and-recent policy scores 0.000 while structure-aware retention scores 1.000 at token granularity and 0.643 at 16-token page granularity ($n=14$ per slice); an adversarial buried-state variant lowers the structure score to 0.429, marking the boundary of the heuristic. Separately, we falsify the hypothesis that soft INT4 storage for 75% of positions creates a meaningful quality separation: on all 240 locked examples, FullKV, role-blind mixed, and structure-aware mixed score 0.8875, 0.8792, and 0.8833. We therefore evaluate the mixed BF16/INT4 path as a systems mechanism only. Our packed cache with FlashInfer-merged decode uses $0.72\times$ measured KV payload bytes and $0.87\times$ peak allocated CUDA memory relative to FullKV, while end-to-end latency is $1.11\text{--}1.12\times$ and time per output token is $1.20\text{--}1.21\times$: the current implementation is a bytes win and a latency cost, and the gap between payload and peak is caused by an explicit INT4-to-BF16 cold-attention scratch buffer. The study supports three scoped conclusions: agent-trace structure is a useful retention prior under aggressive eviction, soft INT4 is quality-neutral at the tested operating point, and packed bytes do not automatically imply peak-memory or latency wins.

1 Introduction

Autoregressive Transformer inference stores the keys and values of previous tokens so that generation does not recompute the complete prefix [1]. The resulting KV cache grows with context length and batch size and is a central memory-management concern in LLM serving [2]. Existing work reduces this cost through paging, eviction, token selection, or quantization [2, 3, 4, 5, 6, 7, 8, 9]. These methods generally estimate token importance from position, attention, or activation statistics.

Agent traces expose another signal before any attention is evaluated: their protocol structure. A serving layer often knows which spans are system instructions, tool schemas, tool results, user constraints, or generated dialogue. A role-blind cache policy can therefore satisfy a nominal token budget while deleting state that is necessary for the agent’s next action, and aggregate language-model metrics need not expose the resulting failures: a malformed tool call, compliance with an obsolete instruction, or use of the wrong order identifier.

PriorityKV asks two questions that must remain separate, because eviction removes information while quantization retains an approximation of it:

1. At a matched eviction budget, does preserving protocol structure improve agent-task reliability over role-blind position policies?

2. If low-precision KV storage is quality-neutral, can a working mixed BF16/INT4 cache still provide measurable systems value?

Our initial expectation was that role-aware INT4 placement would create a quality gap. It did not at the selected 75% INT4 operating point (Section 6.2). We retain that negative result and evaluate the packed path in bytes and latency instead.

Contributions. This report makes four contributions:

1. **PriorityBench-A:** a reproducible 240-example agent-reliability benchmark with deterministic scorers and a frozen manifest hash (Section 3).
2. **Matched-budget eviction evidence:** structure-aware retention outperforms role-blind sink-and-recent, random, and fixed-prefix controls on targeted stress slices, with adversarial buried-state and middle-relocation checks that scope the result (Section 6.1).
3. **A falsified quality hypothesis:** role-blind and structure-aware mixed INT4 policies both remain close to FullKV on the locked 240-example evaluation (Section 6.2).
4. **An auditable systems prototype:** packed BF16/INT4 pages, FlashInfer log-sum-exp state merging, phase-separated latency, and separate reporting of payload and peak memory on H200 (Sections 4.4 and 6.3).

Scope of claims. The report deliberately does *not* claim: paper-grade generalization across LongBench [10] or RULER [11]; a quantization-driven accuracy advantage; a latency or serving-throughput win from the current implementation; a peak-memory collapse (the cold-scratch buffer prevents it); or that the reduced Gemma check reproduces the Qwen results in absolute terms. The decisive eviction slices contain 14–16 examples each and are reported as controlled stress tests, not as a broad benchmark matrix.

Figure 1 summarizes the system. The diagram follows the token-strip and left-to-right cache-workflow conventions common in H2O [5], SnapKV [6], and KIVI [8], but depicts PriorityKV’s distinct input signal and allocation policy: unlike attention-derived selection, PriorityKV begins from message-role metadata already present in the trace.

2 Related Work

KV memory management. PagedAttention reduces fragmentation and enables flexible KV sharing without changing which tokens are retained [2]. StreamingLLM observes the role of attention sinks and combines initial tokens with a recent window [3]. We use this sink-and-recent shape as a deliberately role-blind eviction control, not as a complete reimplement or evaluation of StreamingLLM.

KV eviction and token selection. Scissorhands retains tokens that were previously important [4], while H2O balances heavy hitters with recent tokens [5]. SnapKV estimates prompt importance from an observation window [6], and PyramidKV varies the budget across layers [7]. These methods derive importance from model behavior. PriorityKV instead asks whether application-visible message roles are useful when the workload is an agent trace. The approaches are complementary—structural priors could be combined with attention-based selection—but that combination is not evaluated here, and Section 8 lists the absence of tuned H2O/SnapKV baselines as a limitation.

KV quantization. KIVI motivates asymmetric low-bit treatment of keys and values and demonstrates that aggressive KV quantization can preserve downstream quality [8]. Our INT4 path is not proposed as a replacement for KIVI or as a new quantizer. It is a storage and measurement vehicle for comparing role-aware and role-blind precision allocation under a matched budget.

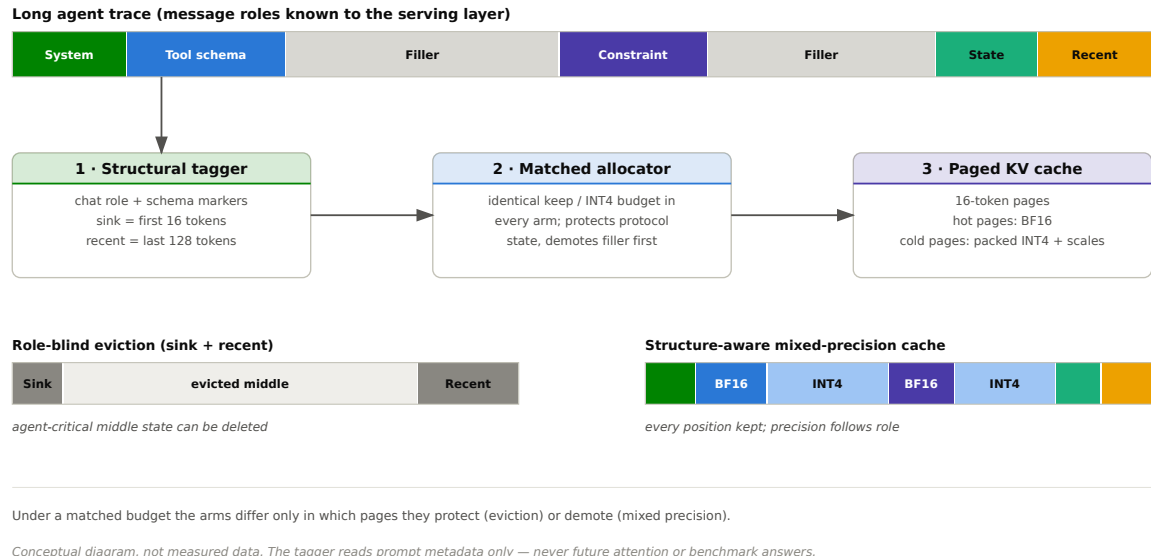


Figure 1: Conceptual overview of PriorityKV (illustration, not measured data). An agent trace arrives with known message roles (top strip). A structural tagger maps spans to page roles and protects a 16-token sink and a 128-token recent window; a matched allocator then applies the same keep or INT4 budget as every baseline arm, differing only in *which* pages it protects; storage is a paged KV cache with BF16 hot pages and packed INT4 cold pages. Bottom left: role-blind sink-and-recent eviction can delete agent-critical middle state. Bottom right: the mixed-precision cache retains every position and lowers precision on filler instead. The tagger uses prompt metadata only—never future attention or benchmark answers.

Serving and evaluation. FlashInfer provides composable attention kernels and state merging for heterogeneous serving workloads [9]. PriorityKV uses its `merge_state` primitive to combine hot BF16 and dequantized cold attention results (Section 4.4). LongBench [10] and RULER [11] provide broad long-context evaluations. PriorityBench-A is narrower: it isolates deterministic agent-state failures and should be read as a complementary diagnostic rather than a general long-context benchmark.

3 PriorityBench-A

3.1 Tasks and construction

PriorityBench-A contains 240 generated multi-turn conversations, with 80 examples in each of three categories (Table 1). Each conversation plants relevant state early, adds benign filler, and ends with a request whose correct answer depends on that state. Scoring is programmatic (0/1 per example) and does not use an LLM-as-judge.

Table 1: The three PriorityBench-A task categories (80 examples each). Every scorer is deterministic; a slice score is the mean of per-example 0/1 outcomes.

Category	Required behavior	Scoring
Tool schema	Emit a valid call under an earlier tool contract	JSON parsing plus schema-subset checks
Instruction supersession	Follow the latest of conflicting constraints	Required and forbidden regular expressions
Multi-turn state	Reuse an earlier ID, path, or preference	Exact slot match

The lock contains 83 examples at 8k, 81 at 16k, and 76 at 32k tokens; split counts are 92 calibration, 49 validation, and 99 test, assigned by stable hashing. The frozen manifest has SHA256

fc44b966725738c94008ba61ce57ad7366169b9c0be73074f8161d909ccfae89.

PriorityBench-A is synthetic by design. It offers controlled failure attribution and exact scoring, but it does not estimate the frequency of these failures in production traffic. The generator, templates, manifest, and scorers are included in the repository.

3.2 Adversarial variants and leakage controls

The initial templates often place short, gold-bearing turns before long filler, so a policy that simply prefers short or early spans could appear semantic without being so. Two controls address this:

- **Buried state:** gold-bearing turns are lengthened or buried in filler so that short-turn tagging no longer identifies all state (40 of the 240 locked examples carry a buried-state flag).
- **Middle relocation:** the state block is moved away from the prefix while preserving the leading system message and final request, separating structural retention from mere early-position retention.

These variants exposed a real limitation: the structure policy preserves tool and constraint roles more reliably than unmarked free-form state (Section 6.1). An early tagger implementation keyed on final-answer markup was removed; the buried-state and middle-relocation controls were added specifically to detect such leakage.

4 Method

4.1 Structural roles

PriorityKV maps chat messages to page roles using information already available to an agent serving stack. System messages are tagged as system, tool, or constraint spans; tool messages and schema-like assistant messages are tagged as tool spans; constraint-like user messages are tagged as constraints; and short state-bearing turns can be tagged as other structured state. Independent of roles, the first 16 tokens are protected as an attention sink and the most recent 128 tokens are protected as a decode-local window.

The tagger is a transparent heuristic over message roles, message length, and a small set of schema and constraint markers. It is not a learned importance model and does not inspect the benchmark answer.

4.2 Matched-budget eviction

For a prompt of n tokens and keep fraction $k \in (0, 1]$, define the nominal retention budget

$$B(n, k) = \text{round}(kn). \quad (1)$$

Let S be the sink positions (first 16 tokens), R the recent window (last 128 tokens), and $M = S \cup R$ the mandatory set shared by every arm. Each arm selects a retained set $A \subseteq \{1, \dots, n\}$ with

$$M \subseteq A, \quad |A| = \max(B(n, k), |M|), \quad (2)$$

so all arms receive exactly the same budget and differ only in how they fill $A \setminus M$:

- **Role-blind sink+recent:** fill the budget backward from the recent edge (the StreamingLLM-shaped control).
- **Random:** sample the remaining positions or pages uniformly.
- **Fixed prefix:** take the lowest-index pages, a control that distinguishes structural retention from early-position retention.
- **Structure-aware:** retain protected-role pages first, then fill any remaining budget near the recent edge.

- **Keep-all:** retain the complete prompt, as a wiring control.

Page-granularity experiments operate on contiguous pages of $P=16$ tokens. Token-granularity selection gathers the retained positions and regenerates the cache so that positions are re-indexed contiguously, avoiding an invalid RoPE index substitution. These eviction experiments isolate missing-state sensitivity; they are not the packed mixed-precision serving path of Section 4.3.

The role-blind sink-and-recent arm is expected to fail whenever required state sits in the evicted middle; the informative comparisons are therefore structure against the random and fixed-prefix controls at the same budget, and the adversarial variants that probe what the heuristic misses.

4.3 Matched mixed precision

The mixed-precision policy assigns the same number of INT4 positions in both arms. With target fraction $f = 0.75$ of the cache, sink and recent positions always remain BF16, and both arms demote exactly the same count N_4 of positions (the realized fraction in the frozen run is 0.750). The role-blind arm spaces INT4 positions uniformly through the demotable region; the structure arm demotes filler and low-risk roles first, spilling into protected roles only when required to meet the exact budget.

For the frozen Qwen3-8B path, K and V are divided into 16-token pages. Hot pages store BF16 tensors; cold pages store packed 4-bit values (two per byte) plus per-group scale metadata. Packing is real storage, not a fake-quantized BF16 tensor. Byte accounting reports both the actual payload (packed values plus metadata) and an idealized 0.5-byte-per-element model, and the two are deliberately kept distinct in Section 6.3.

4.4 FlashInfer decode with LSE merging

Prefill uses the native Hugging Face path. The cache is then split into hot BF16 pages and cold packed pages. During decode, the Qwen3 attention shim performs at most two attention calls per layer: one over the coalesced hot pages and the decode tail, and one over cold pages after dequantization. Each call returns a partial attention output with its log-sum-exp (LSE) state; FlashInfer’s `merge_state` combines the two partial softmax states into the exact attention result over the union of pages. Parity tests against a monolithic reference reduced the maximum absolute merge error to approximately 4.88×10^{-4} after correcting an LSE-base mismatch.

The current implementation expands cold INT4 pages into a BF16 GPU scratch buffer before the cold attention call (Figure 2). This avoids silently materializing a full Hugging Face cache, but it limits peak memory savings and adds decode overhead; it is the single mechanism behind both the payload-versus-peak gap and the TPOT regression in Section 6.3. Direct fused or page-streamed INT4 attention is future systems work.

5 Experimental Setup

The primary experiments use Qwen3-8B [12] in non-thinking greedy decode mode on NVIDIA H200 GPUs, at the pinned model revision recorded in `FINAL_RUN_MANIFEST.yaml` together with the benchmark hash, frozen configurations, job IDs, and pass/fail decisions. The evaluation families answer different questions and have deliberately different sizes (Table 2).

Table 2: Evaluation families. The decisive eviction slices are small by design (controlled stress tests); the 240-example lock measures mixed INT4 quality, not eviction.

Family	Size	Purpose
Matched-keep stress	14–16 per configuration	Controlled eviction sensitivity and policy ablations
Lock-240 mixed quality	240	Quality of FullKV and matched 75% INT4 placement
D4 latency	18 (9 per 8k/16k stratum)	Pack, cold-scratch, TTFT, and TPOT measurement
Peak and payload	18	Allocated/reserved CUDA peak and cache payload bytes
Gemma reduced	14 at $\approx 8k$	Directional second-model check only

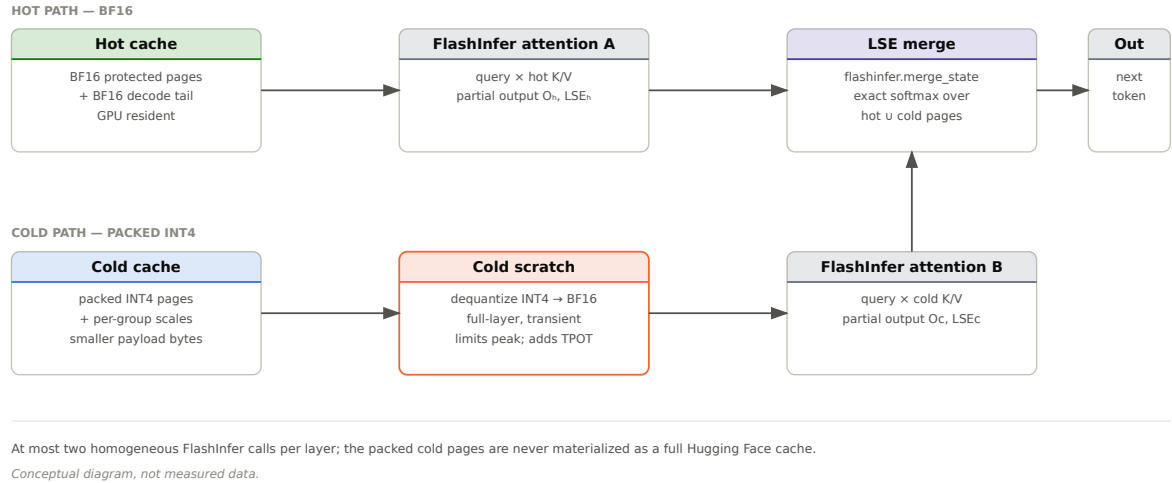


Figure 2: Conceptual dataflow of the mixed-precision decode path (illustration, not measured data). Hot BF16 pages and the decode tail feed one FlashInfer attention call; cold packed INT4 pages are first dequantized into a BF16 scratch buffer and feed a second call; `merge_state` combines the two log-sum-exp states into the exact softmax over all pages (maximum absolute parity error $\approx 4.88 \times 10^{-4}$). The scratch buffer is drawn explicitly because it is the mechanism that limits peak memory to $0.87\times$ FullKV despite a $0.72\times$ payload, and it contributes to the $\sim 1.20\times$ TPOT cost (Section 6.3).

Latency rows use 128 generated tokens, greedy decoding, one warm-up sequence, and three timing repeats whose median is recorded. FullKV uses the SDPA attention path; mixed arms use the FlashInfer shim, so the latency comparison includes the backend difference and is not kernel-isolated. This is a single-request prototype comparison, not a throughput or concurrency study. An FP8 baseline job uses a different batch-amortized timing protocol and is therefore retained as a pass/fail secondary check rather than merged into the main latency table.

6 Results

6.1 Structure matters under matched-budget eviction

Table 3 reports the matched-budget eviction slices and Figure 3 plots the page-budget sweep. At a 25% token keep budget, the role-blind sink-and-recent arm scores 0.000 while structure-aware retention scores 1.000 (14/14). At page granularity the structure arm reaches 0.643 (9/14) and stays on that plateau across the 15–35% budget sweep, while the random control climbs from 0.071 to 0.429 as the budget grows.

Table 3: Matched-budget eviction stress on Qwen3-8B. All arms in a row receive the same budget; “—” means the control was not run in that slice. Scores are means of deterministic 0/1 outcomes; at these sample sizes one example changes a score by ~ 0.06 – 0.07 . The same 14-example slice is reused across the page-budget sweep, so those rows are an ablation over one fixed slice, not independent replications.

Setting	n	Keep-all	Role-blind	Random	Fixed prefix	Structure
Token keep, $k=0.25$	14	1.000	0.000	0.000	—	1.000
Page keep, $k=0.15$	14	1.000	0.000	0.071	—	0.643
Page keep, $k=0.25$	14	1.000	0.000	0.286	—	0.643
Page keep, $k=0.35$	14	1.000	0.000	0.429	—	0.643
Buried token state, $k=0.25$	14	1.000	0.000	0.000	—	0.429
Middle-relocated pages, $k=0.25$	16	1.000	0.000	—	0.125	0.688

Three observations follow. First, the role-blind zero is expected—a sink-and-recent window necessarily deletes required middle state at these budgets—and it quantifies how brittle that default shape is on agent traces. Second, the structure arm’s 0.643 plateau and its drop to 0.429 (6/14) under buried state are equally important: explicit roles rescue a subset of failures, but the heuristic does not identify all free-form multi-turn state. Third, middle relocation separates structure (0.688, 11/16) from the fixed-prefix control (0.125, 2/16), showing that the effect is not merely early-position retention.

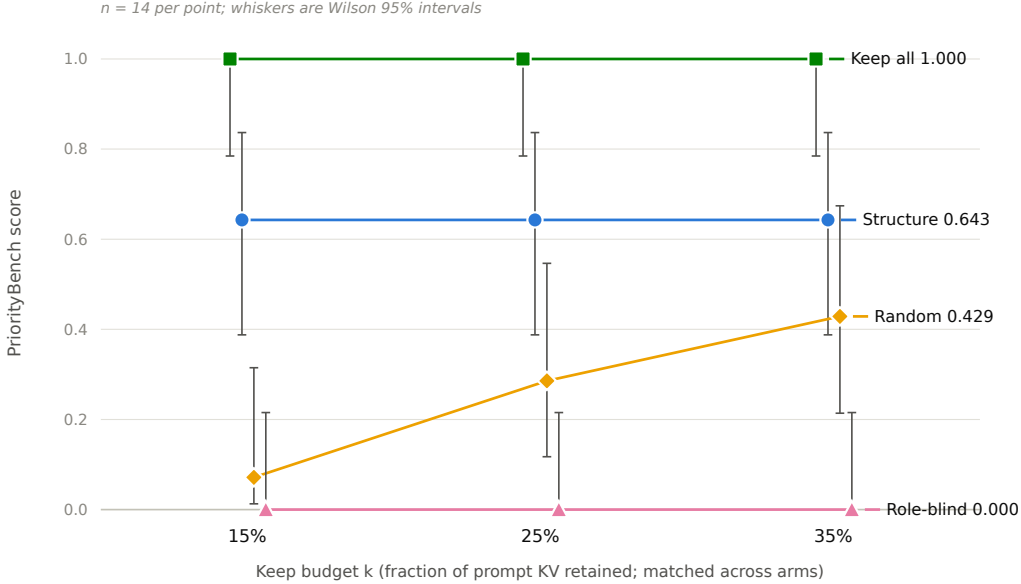


Figure 3: Measured score versus matched keep budget k for page-granularity eviction (Qwen3-8B, 16-token pages, same fixed 14-example stress slice at every budget). Role-blind sink-and-recent stays at 0.000 across budgets; random retention climbs with budget (0.071 $\rightarrow$ 0.429); structure-aware retention holds a 0.643 plateau; keep-all confirms 1.000 wiring. Whiskers are Wilson 95% intervals for $n=14$ and points are dodged horizontally for legibility. The sweep is an ablation over one slice, not three independent replications (Table 3).

6.2 Soft INT4 does not create a quality separation

Table 4 reports the locked 240-example mixed-precision evaluation and Figure 4 breaks it down by context length. The structure and role-blind arms differ by 0.0042, equivalent to one aggregate success on this benchmark; we do not interpret that difference as evidence of a structure-aware quantization quality advantage. This is the falsification of our starting hypothesis, and we report it as such.

Table 4: Locked 240-example quality at matched INT4 budget (`int4_frac`=0.75; realized fraction 0.750), Qwen3-8B. Per-stratum sizes: $n=83$ (8k), 81 (16k), 76 (32k). The 0.0042 gap between the mixed arms equals one example and is not treated as evidence.

Arm	Mean ($n=240$)	Δ vs. FullKV	8k	16k	32k
FullKV	0.8875	—	1.000	1.000	0.645
Role-blind mixed INT4	0.8792	−0.0083	1.000	1.000	0.618
Structure-aware mixed INT4	0.8833	−0.0042	1.000	1.000	0.632

Two secondary observations bound the finding. At 8k and 16k, all three arms score 1.000, so those strata cannot separate any of the arms and the benchmark’s discriminative power for quantization lives entirely in the 32k stratum, where FullKV itself falls to 0.645 and the mixed arms score 0.618 and 0.632. And the

contrast with Section 6.1 is the substantive conclusion: retaining an INT4 *approximation* of 75% of positions is far less destructive than *evicting* those positions, which is why eviction and quantization must not be pooled into a single “compression quality” claim.

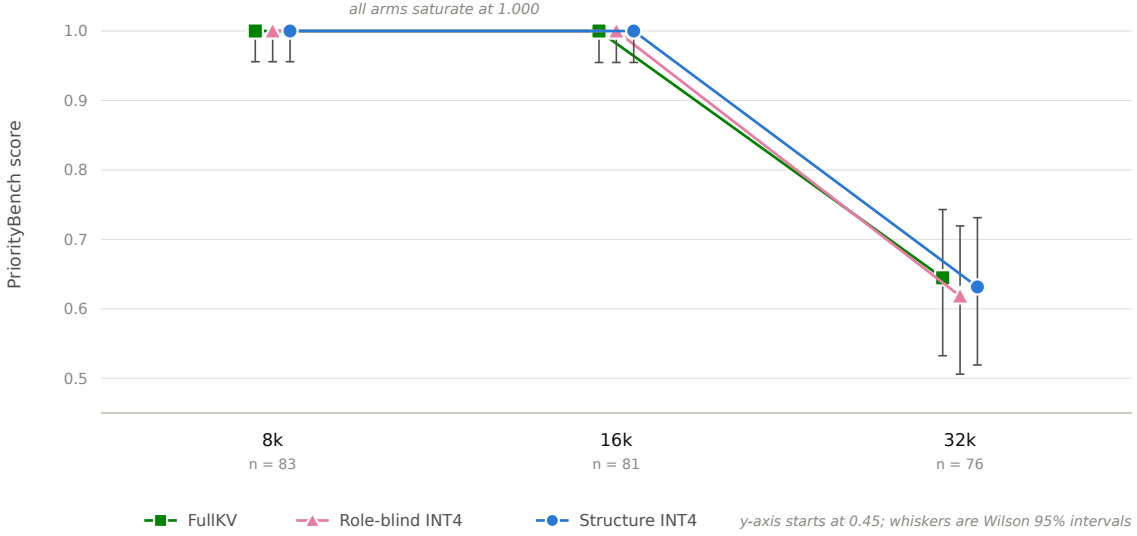


Figure 4: Measured lock-240 quality by context length (Qwen3-8B, `int4_frac=0.75`; $n=83/81/76$ at 8k/16k/32k; y-axis starts at 0.45). All three arms are saturated at 1.000 for 8k and 16k; only the 32k stratum separates FullKV (0.645) from role-blind (0.618) and structure-aware (0.632) mixed INT4, and the Wilson 95% intervals of the three arms overlap there, visually supporting the no-separation conclusion. Points are dodged horizontally for legibility. Overall means: 0.8875, 0.8792, 0.8833 ($n=240$); the arm differences correspond to one or two examples.

6.3 Packed storage saves bytes, costs latency

Table 5 reports the phase-separated latency measurement and Figure 5 summarizes the memory and latency ratios together.

Table 5: Single-request latency on H200 (Qwen3-8B; 128 generated tokens; greedy; median of three repeats; 9 examples per stratum). FullKV runs SDPA; the structure arm runs the packed cache with the FlashInfer shim, so ratios include the backend difference. Pack and cold-scratch are one-time per-sequence costs. The mixed path is a per-token slowdown, not a speedup.

Context	TPOT (ms)		Pack (ms)	Cold scratch (ms)	E2E ratio	TPOT ratio	Score
	FullKV	Structure					
8k	25.25	30.29	34.8	14.2	1.118	1.200	1.000
16k	23.84	28.95	48.1	19.7	1.113	1.215	0.889

The mixed path is close to FullKV end-to-end ($1.11\text{--}1.12\times$) because prefill dominates these single-request traces, but it is approximately 20% slower per output token; the decode-only time-to-first-token ratio is $1.17\text{--}1.19\times$. Packing and cold-scratch construction cost tens of milliseconds per sequence. We state this plainly: the current implementation is a latency cost, not a speedup, and the measurements exist so that the cost is not hidden behind the bytes saving.

Across the 18-example memory slice, FullKV allocates 23.61 GB at peak and the structure-aware mixed path allocates 20.50 GB, a ratio of 0.868. Actual mixed payload is 1.679 GB against a 2.336 GB BF16 cache model, a ratio of 0.719; the idealized 4-bit model gives 0.473. The three ratios answer different questions

and must not be substituted for one another: the modeled ratio is the ceiling a fused implementation could approach, the payload ratio is what the packed storage actually holds (including scale metadata), and the peak ratio is what the allocator actually experienced, inflated by the BF16 cold scratch of Section 4.4.

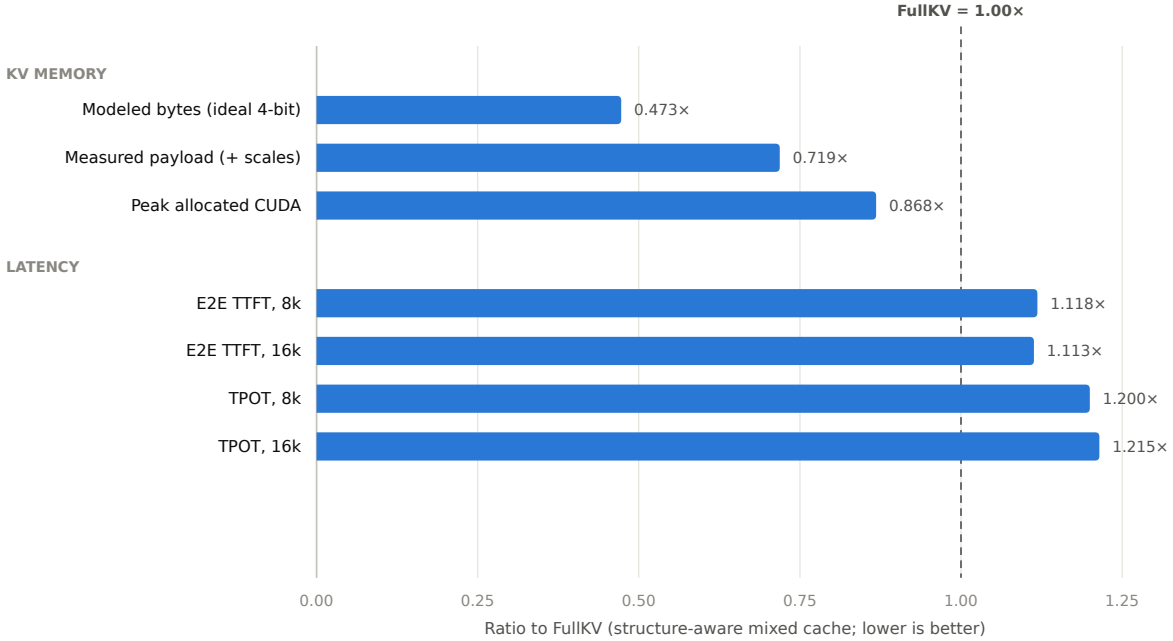


Figure 5: Measured systems ratios for the structure-aware mixed cache, normalized to FullKV on H200 (18-example slices; the dashed vertical line marks FullKV parity). The three memory ratios are distinct quantities that must not be substituted for one another: modeled bytes (0.473×) are the idealized 4-bit ceiling, measured payload including scale metadata is 0.719×, and peak allocated CUDA memory is 0.868×; the payload-to-peak gap is caused by the INT4-to-BF16 cold-attention scratch. End-to-end latency (1.113–1.118×) and time per output token (1.200–1.215×) sit right of parity—a measured cost, reported as such.

6.4 Secondary model check

On a reduced 14-example Gemma-2-9B-IT check at approximately 8k context with $k=0.25$, FullKV, structure-aware, and role-blind eviction score 0.357, 0.143, and 0.000. This supports only the direction $\text{structure} \geq \text{role-blind}$ under the tested stress. The absolute values are not comparable to the Qwen lock: the prompt and scorer design and the context cap were tuned around Qwen3-8B, and FullKV itself scores only 0.357 on this slice.

7 Discussion

The experiments support a narrow design principle: when an application already knows the protocol role of a span, discarding that information before cache allocation is unnecessary. Message roles are cheap, deterministic priors that can protect tool and instruction state when attention-based importance estimates are unavailable or stale.

The results also show why eviction and quantization should not be grouped under a single “compression quality” claim. In our setup, evicting 75% of positions destroys the targeted agent behaviors (Table 3), while storing approximately 75% of positions in INT4 leaves the aggregate result near FullKV (Table 4). A mixed-precision placement policy should therefore be justified by memory and kernel behavior unless a harder quality regime demonstrates otherwise.

Finally, payload accounting is not a proxy for deployed memory or performance. The prototype stores fewer bytes but re-expands cold pages for attention. A production implementation would need fused low-bit attention or bounded page streaming, allocator analysis under concurrency, and integration into a serving scheduler before claiming capacity or throughput gains.

8 Limitations and Threats to Validity

1. **Synthetic benchmark.** PriorityBench-A provides controlled, deterministic failures but is not sampled from production agent traffic.
2. **Small positive-result slices.** The decisive eviction experiments use 14–16 examples, and the page-budget sweep reuses one fixed slice. The 240-example run evaluates mixed INT4 quality, not matched eviction.
3. **One primary model and device.** Qwen3-8B on H200 is the only complete matrix. The Gemma check is reduced, low-scoring, and directional only.
4. **Heuristic structure labels.** Free-form state without recognizable roles remains a failure mode (the buried-state score of 0.429). Structural metadata can also be missing or wrong in real systems.
5. **Limited baselines.** Role-blind sink/recent, random, fixed-prefix, an FP8 check, and a reduced SnapKV attempt do not constitute a comparison with tuned H2O, SnapKV, PyramidKV, or state-of-the-art KV quantizers.
6. **Ceiling effects.** All lock-240 arms score 1.000 at 8k and 16k, so the quantization comparison is informative only in the 32k stratum.
7. **Prototype timing.** Measurements cover single-request latency, not batching, concurrency, throughput, or tail latency; FullKV SDPA versus mixed FlashInfer is not a kernel-isolated comparison.
8. **Cold scratch.** INT4-to-BF16 expansion limits the realized peak-memory reduction and drives the measured TPOT regression.
9. **No cross-run uncertainty.** Per-example timing medians reduce local noise, but canonical jobs were not repeated across independent machine runs. Figure whiskers are Wilson 95% intervals for within-slice binomial uncertainty only; they do not capture run-to-run or slice-to-slice variance.

PriorityBench-A contains generated tool contracts and synthetic identifiers rather than personal data. The main foreseeable misuse of this report is overgeneralizing the benchmark to claim production reliability; the frozen claim boundary and these limitations are intended to prevent that reading.

9 Reproducibility

The repository (https://github.com/Arush777/Priority_KV) records the benchmark generator, templates, scorers, manifest, and SHA256 audit; the exact Qwen3-8B revision and non-thinking generation settings; frozen YAML configurations and job commands; raw summaries, logs, and GPU snapshots for the canonical jobs; CPU unit tests for policies, packed storage, quantization, page management, and FlashInfer state logic; and `FINAL_RUN_MANIFEST.yaml` as the claim and artifact index. Table 6 maps each headline claim to its tracked result bundle.

Generate and audit the locked dataset with:

```
PYTHONPATH=src uv run python scripts/mk_bench.py --mode w3_lock
PYTHONPATH=src uv run python scripts/audit_bench.py
```

and regenerate every paper figure from the frozen artifacts with:

```
uv run python scripts/make_publication_figures.py
```

Table 6: Claim-to-artifact index. Each job ID names a tracked bundle under `jobs/results/` containing the machine-readable summary, logs, and GPU snapshots.

Claim	Canonical job
Latency (Table 5)	d4_latency_m3c_gpu56_r1
Peak and payload (Figure 5)	mg_a_peak_mem_gpu5_r1
Lock-240 quality (Table 4)	mg_b_lock240_quality_gpu01_r1
Gemma reduced (Section 6.4)	pub_c_gemma_reduced_gpu01_r6

GPU re-execution requires the pinned model weights, the `gpu` optional dependency set, and an H200-class CUDA environment; canonical commands and bundle formats are documented under `jobs/` and in `docs/REPRODUCIBILITY.md`.

10 Conclusion

PriorityKV demonstrates that known agent-trace structure is a useful cache-allocation signal under aggressive eviction. It does not demonstrate that role-aware soft INT4 placement improves quality, and it does not hide that the current packed implementation is slower per decoded token. The complete result is a measurement result rather than a universal cache-compression claim: preserve protocol-critical state when dropping KV, use low precision for bytes when quality permits, and report payload, peak, and latency as separate quantities.

References

- [1] A. Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.
- [2] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023. <https://arxiv.org/abs/2309.06180>
- [3] G. Xiao et al. Efficient streaming language models with attention sinks. In *ICLR*, 2024. <https://arxiv.org/abs/2309.17453>
- [4] Z. Liu et al. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *NeurIPS*, 2023. <https://arxiv.org/abs/2305.17118>
- [5] Z. Zhang et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. <https://arxiv.org/abs/2306.14048>
- [6] Y. Li et al. SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*, 2024. <https://arxiv.org/abs/2404.14469>
- [7] Z. Cai et al. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. arXiv preprint, 2024. <https://arxiv.org/abs/2406.02069>
- [8] Z. Liu et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *ICML*, 2024. <https://arxiv.org/abs/2402.02750>
- [9] Z. Ye et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving. arXiv preprint, 2025. <https://arxiv.org/abs/2501.01005>
- [10] Y. Bai et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *ACL*, 2024. <https://arxiv.org/abs/2308.14508>
- [11] C.-P. Hsieh et al. RULER: What’s the real context size of your long-context language models? In *COLM*, 2024. <https://arxiv.org/abs/2404.06654>
- [12] A. Yang et al. Qwen3 technical report. arXiv preprint, 2025. <https://arxiv.org/abs/2505.09388>